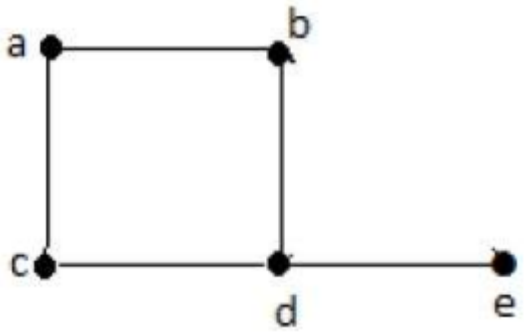
Graphs

In the domain of mathematics and computer science, graph theory is the study of graphs that concerns with the relationship among edges and vertices. It is a popular subject having its applications in computer science, information technology, biosciences, mathematics, and linguistics to name a few.

What is a Graph?

A graph is a pictorial representation of a set of objects where some pairs of objects are connected by links. The interconnected objects are represented by points termed as **vertices,** and the links that connect the vertices are called **edges**.

Formally, a graph is a pair of sets V, E, where **V** is the set of vertices and **E** is the set of edges, connecting the pairs of vertices. Take a look at the following graph −



In the above graph,

V = {a, b, c, d, e}

E = {ab, ac, bd, cd, de}

Applications of Graph Theory

Graph theory has its applications in diverse fields of engineering −

Electrical Engineering − The concepts of graph theory is used extensively in designing circuit connections. The types or organization of connections are named as topologies. Some examples for topologies are star, bridge, series, and parallel topologies.

Computer Science − Graph theory is used for the study of algorithms. For example,

Kruskal's Algorithm

Prim's Algorithm

Dijkstra's Algorithm

Computer Network − The relationships among interconnected computers in the network follows the principles of graph theory.

Science − The molecular structure and chemical structure of a substance, the DNA structure of an organism, etc., are represented by graphs.

Linguistics − The parsing tree of a language and grammar of a language uses graphs.

General − Routes between the cities can be represented using graphs. Depicting hierarchical ordered information such as family tree can be used as a special type of graph called tree.

GRAPH THEORY - FUNDAMENTALS

A graph is a diagram of points and lines connected to the points. It has at least one line joining a set of two vertices with no vertex connecting itself. The concept of graphs in graph theory stands up on some basic terms such as point, line, vertex, edge, degree of vertices, properties of graphs, etc. Here, in this chapter, we will cover these fundamentals of graph theory.

Point

A point is a particular position in a one-dimensional, two-dimensional, or three-dimensional space. For better understanding, a point can be denoted by an alphabet. It can be represented with a dot.

Example

Here, the dot is a point named 'a'.

Line

A **Line** is a connection between two points. It can be represented with a solid line.

Example

Here, 'a' and 'b' are the points. The link between these two points is called a line.

Vertex

A vertex is a point where multiple lines meet. It is also called a **node**. Similar to points, a vertex is also denoted by an alphabet.

Example

Here, the vertex is named with an alphabet 'a'.

Edge

An edge is the mathematical term for a line that connects two vertices. Many edges can be formed from a single vertex. Without a vertex, an edge cannot be formed. There must be a starting vertex and an ending vertex for an edge.

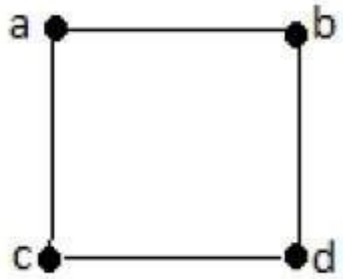
Example

Here, ‘a’ and ‘b’ are the two vertices and the link between them is called an edge.

Graph

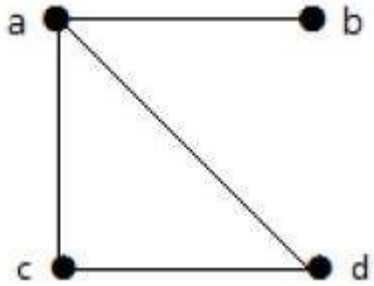
A graph ‘G’ is defined as G $= V, E$ Where V is a set of all vertices and E is a set of all edges in the graph.

Example 1



In the above example, ab, ac, cd, and bd are the edges of the graph. Similarly, a, b, c, and d are the vertices of the graph.

Example 2



In this graph, there are four vertices a, b, c, and d, and four edges ab, ac, ad, and cd.

Loop

In a graph, if an edge is drawn from vertex to itself, it is called a loop.

Example 1

In the above graph, V is a vertex for which it has an edge V, V forming a loop.

Example 2

In this graph, there are two loops which are formed at vertex a, and vertex b.

Degree of Vertex

It is the number of vertices incident with the vertex V.

Notation − deg$V$.

In a simple graph with n number of vertices, the degree of any vertices is −

```
deg(v) ≤ n - 1 ∀ v ∈ G
```

A vertex can form an edge with all other vertices except by itself. So the degree of a vertex will be up to the **number of vertices in the graph minus 1**. This 1 is for the self-vertex as it cannot form a loop by itself. If there is a loop at any of the vertices, then it is not a Simple Graph.

Degree of vertex can be considered under two cases of graphs −

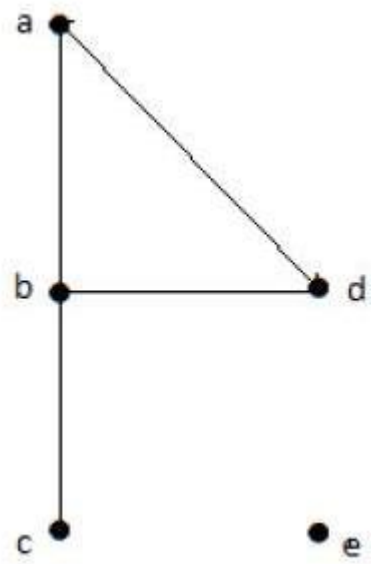
- Undirected Graph
- Directed Graph

Degree of Vertex in an Undirected Graph

An undirected graph has no directed edges. Consider the following examples.

Example 1

Take a look at the following graph −



In the above Undirected Graph,

- deg$a$ = 2, as there are 2 edges meeting at vertex 'a'.
- deg$b$ = 3, as there are 3 edges meeting at vertex 'b'.
- deg$c$ = 1, as there is 1 edge formed at vertex 'c'

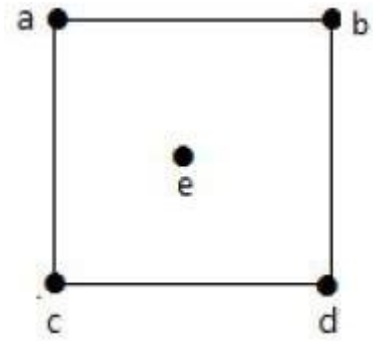
  So 'c' is a **pendent vertex**.
- deg$d$ = 2, as there are 2 edges meeting at vertex 'd'.

- deg$e$ = 0, as there are 0 edges formed at vertex ‘e’.

So ‘e’ is an **isolated vertex**.

Example 2

Take a look at the following graph −



In the above graph,

deg$a$ = 2, deg$b$ = 2, deg$c$ = 2, deg$d$ = 2, and deg$e$ = 0.

The vertex ‘e’ is an isolated vertex. The graph does not have any pendent vertex.

Degree of Vertex in a Directed Graph

In a directed graph, each vertex has an **indegree** and an **outdegree**.

Indegree of a Graph

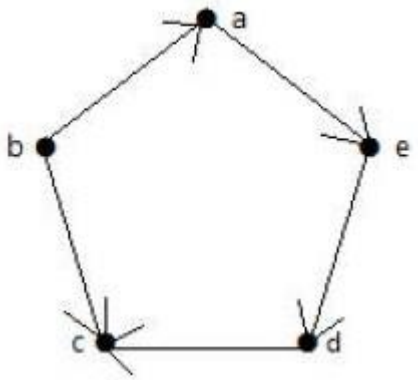
- Indegree of vertex V is the number of edges which are coming into the vertex V.

Outdegree of a Graph

- Outdegree of vertex V is the number of edges which are going out from the vertex V.

Example

Take a look at the following directed graph. Vertex ‘a’ has an edge ‘ae’ going outwards from vertex ‘a’. Hence its outdegree is 1. Similarly, the graph has an edge ‘ba’ coming towards vertex ‘a’. Hence the indegree of ‘a’ is 1.



The indegree and outdegree of other vertices are shown in the following table −

Vertex	Indegree	Outdegree
a	1	1
b	0	2
c	2	0
d	1	1
e	1	1

Pendent Vertex

By using degree of a vertex, we have a two special types of vertices. A vertex with degree one is called a pendent vertex.

Example

Here, in this example, vertex ‘a’ and vertex ‘b’ have a connected edge ‘ab’. So with respect to the vertex ‘a’, there is only one edge towards vertex ‘b’ and similarly with respect to the vertex ‘b’, there is only one edge towards vertex ‘a’. Finally, vertex ‘a’ and vertex ‘b’ has degree as one which are also called as the pendent vertex.

Isolated Vertex

A vertex with degree zero is called an isolated vertex.

Example

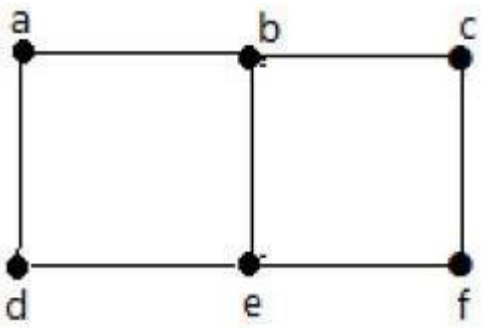
Here, the vertex ‘a’ and vertex ‘b’ has a no connectivity between each other and also to any other vertices. So the degree of both the vertices ‘a’ and ‘b’ are zero. These are also called as isolated vertices.

Adjacency

Here are the norms of adjacency −

- In a graph, two vertices are said to be **adjacent,** if there is an edge between the two vertices. Here, the adjacency of vertices is maintained by the single edge that is connecting those two vertices.
- In a graph, two edges are said to be adjacent, if there is a common vertex between the two edges. Here, the adjacency of edges is maintained by the single vertex that is connecting two edges.

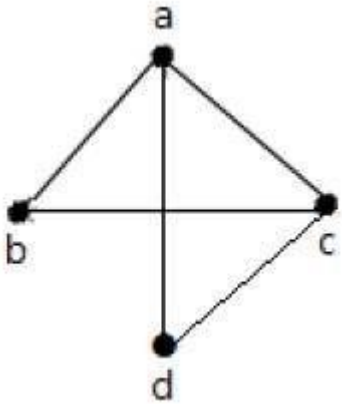
Example 1



In the above graph −

- ‘a’ and ‘b’ are the adjacent vertices, as there is a common edge ‘ab’ between them.
- ‘a’ and ‘d’ are the adjacent vertices, as there is a common edge ‘ad’ between them.
- ab’ and ‘be’ are the adjacent edges, as there is a common vertex ‘b’ between them.
- be’ and ‘de’ are the adjacent edges, as there is a common vertex ‘e’ between them.

Example 2

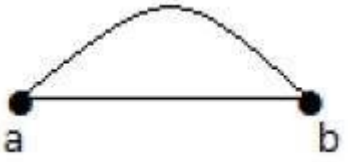


In the above graph −

- a’ and ‘d’ are the adjacent vertices, as there is a common edge ‘ad’ between them.
- ‘c’ and ‘b’ are the adjacent vertices, as there is a common edge ‘cb’ between them.
- ‘ad’ and ‘cd’ are the adjacent edges, as there is a common vertex ‘d’ between them.
- ac’ and ‘cd’ are the adjacent edges, as there is a common vertex ‘c’ between them.

Parallel Edges

In a graph, if a pair of vertices is connected by more than one edge, then those edges are called parallel edges.

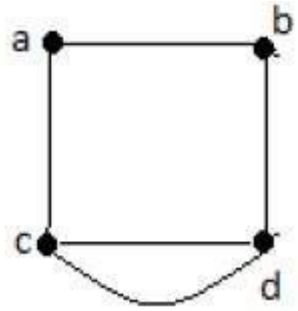


In the above graph, ‘a’ and ‘b’ are the two vertices which are connected by two edges ‘ab’ and ‘ab’ between them. So it is called as a parallel edge.

Multi Graph

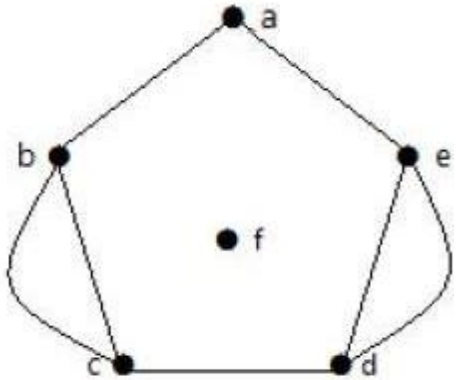
A graph having parallel edges is known as a Multigraph.

Example 1



In the above graph, there are five edges ‘ab’, ‘ac’, ‘cd’, ‘cd’, and ‘bd’. Since ‘c’ and ‘d’ have two parallel edges between them, it a Multigraph.

Example 2



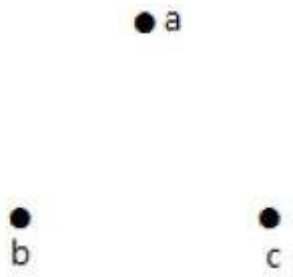
In the above graph, the vertices ‘b’ and ‘c’ have two edges. The vertices ‘e’ and ‘d’ also have two edges between them. Hence it is a Multigraph.

TYPES OF GRAPHS

Null Graph

A **graph having no edges** is called a Null Graph.

Example



In the above graph, there are three vertices named ‘a’, ‘b’, and ‘c’, but there are no edges among them. Hence it is a Null Graph.

Trivial Graph

A **graph with only one vertex** is called a Trivial Graph.

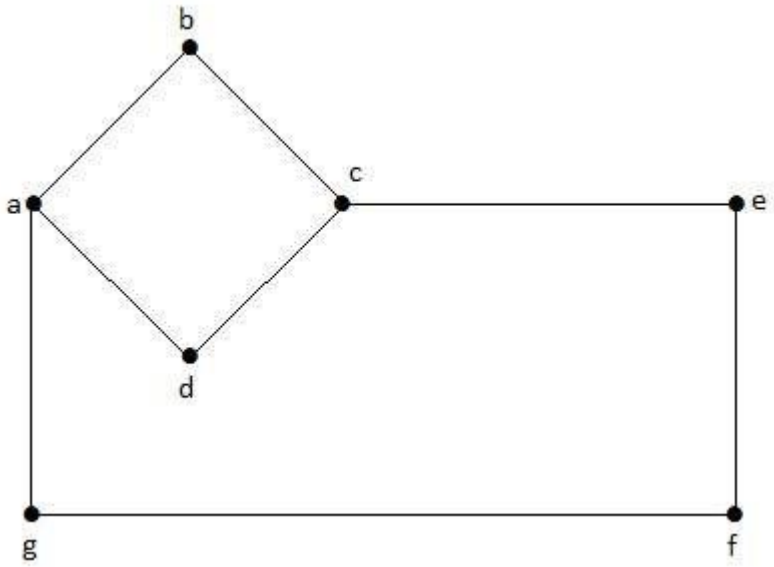
Example

In the above shown graph, there is only one vertex ‘a’ with no other edges. Hence it is a Trivial graph.

Non-Directed Graph

A non-directed graph contains edges but the edges are not directed ones.

Example

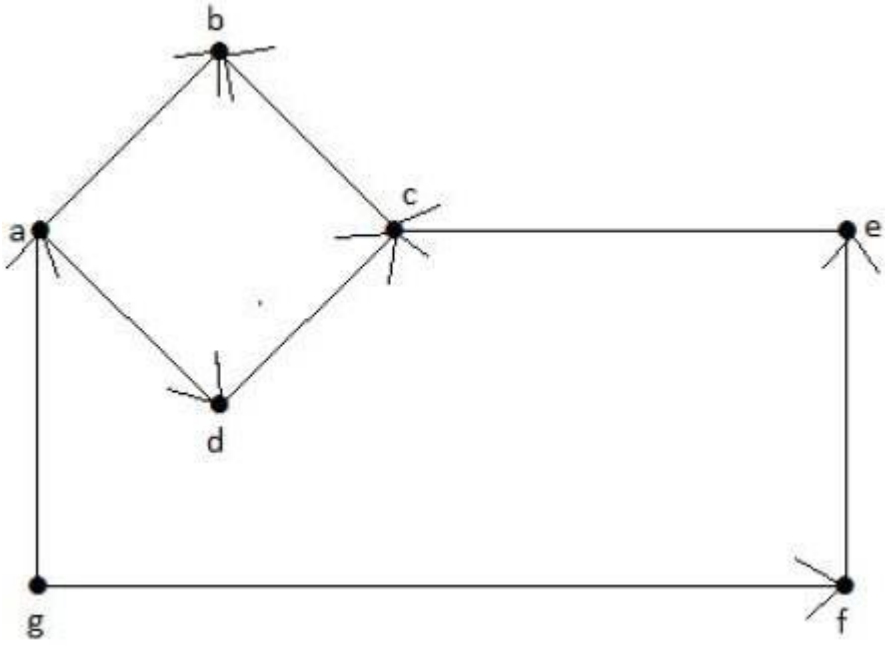


In this graph, ‘a’, ‘b’, ‘c’, ‘d’, ‘e’, ‘f’, ‘g’ are the vertices, and ‘ab’, ‘bc’, ‘cd’, ‘da’, ‘ag’, ‘gf’, ‘ef’ are the edges of the graph. Since it is a non-directed graph, the edges ‘ab’ and ‘ba’ are same. Similarly other edges also considered in the same way.

Directed Graph

In a directed graph, each edge has a direction.

Example



In the above graph, we have seven vertices ‘a’, ‘b’, ‘c’, ‘d’, ‘e’, ‘f’, and ‘g’, and eight edges ‘ab’, ‘cb’, ‘dc’, ‘ad’, ‘ec’, ‘fe’, ‘gf’, and ‘ga’. As it is a directed graph, each edge bears an arrow mark that shows its direction. Note that in a directed graph, ‘ab’ is different from ‘ba’.

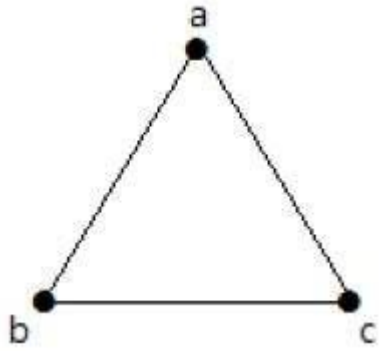
Simple Graph

A graph **with no loops** and **no parallel edges** is called a simple graph.

- The maximum number of edges possible in a single graph with ‘n’ vertices is ${}^nC_2$ where ${}^nC_2 = n(n-1)/2$.
- The number of simple graphs possible with ‘n’ vertices $= 2^{{}^nC_2} = 2^{n(n-1)/2}$.

Example

In the following graph, there are 3 vertices with 3 edges which is maximum excluding the parallel edges and loops. This can be proved by using the above formulae.



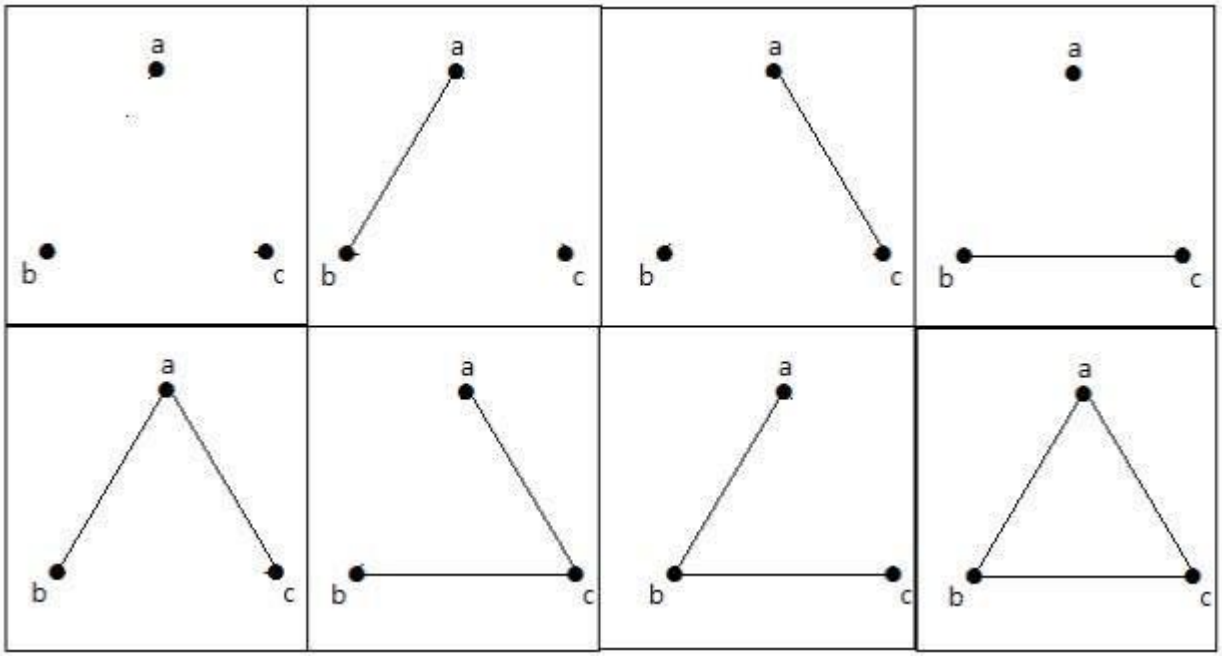
The maximum number of edges with n=3 vertices −

```
nC2 = n(n-1)/2
    = 3(3-1)/2
    = 6/2
    = 3 edges
```

$$2^{{}^{n}C_2} = 2^{n(n-1)/2}$$
$$= 2^{3(3-1)/2}$$
$$= 2^3$$
$$= 8$$

The maximum number of simple graphs with n=3 vertices −

These 8 graphs are as shown below −

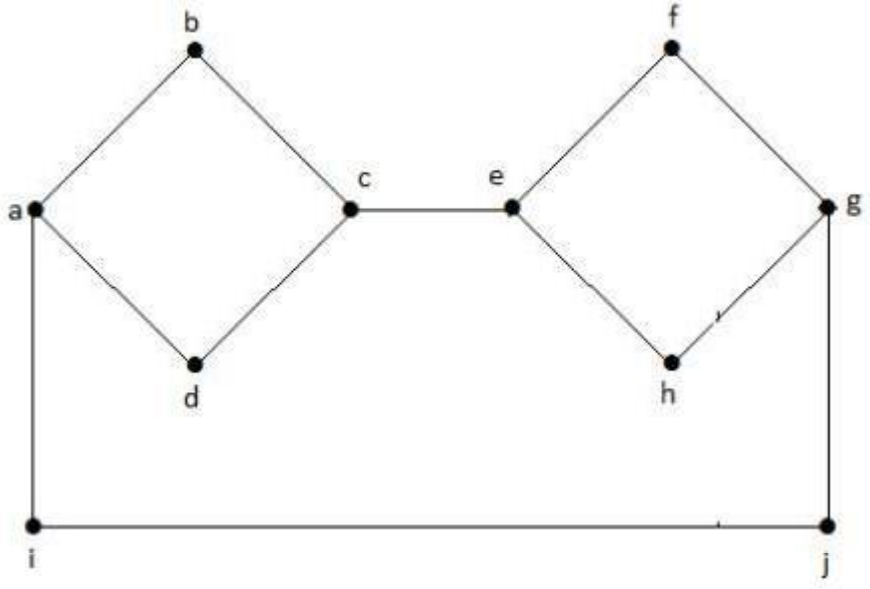


Connected Graph

A graph G is said to be connected **if there exists a path between every pair of vertices**. There should be at least one edge for every vertex in the graph. So that we can say that it is connected to some other vertex at the other side of the edge.

Example

In the following graph, each vertex has its own edge connected to other edge. Hence it is a connected graph.

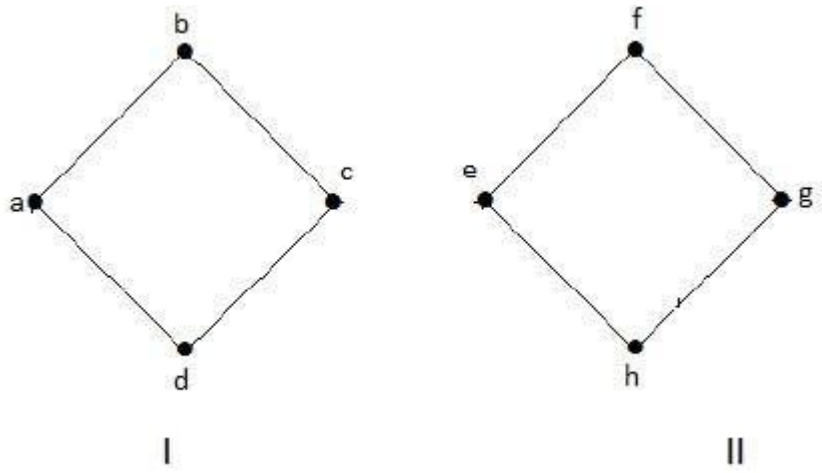


Disconnected Graph

A graph G is disconnected, if it does not contain at least two connected vertices.

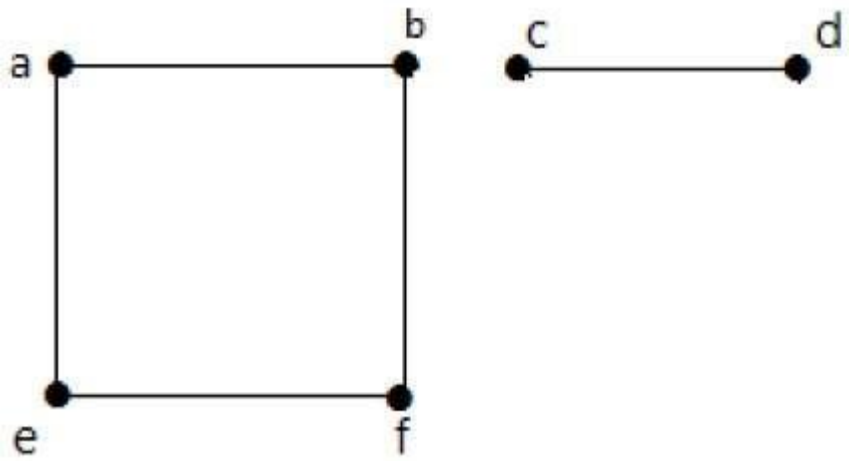
Example 1

The following graph is an example of a Disconnected Graph, where there are two components, one with ‘a’, ‘b’, ‘c’, ‘d’ vertices and another with ‘e’, ’f’, ‘g’, ‘h’ vertices.



The two components are independent and not connected to each other. Hence it is called disconnected graph.

Example 2



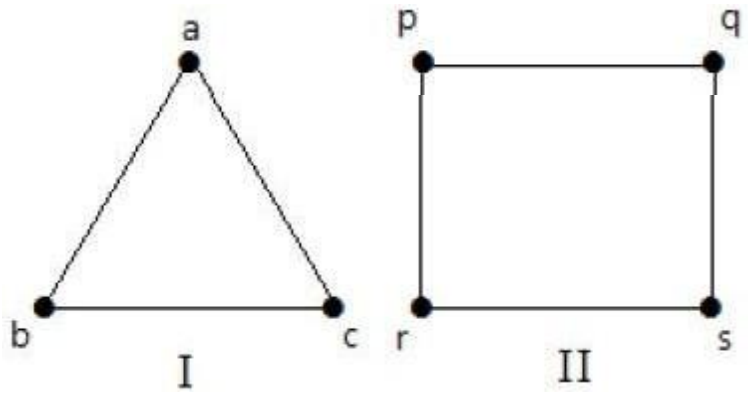
In this example, there are two independent components, a-b-f-e and c-d, which are not connected to each other. Hence this is a disconnected graph.

Regular Graph

A graph G is said to be regular, **if all its vertices have the same degree**. In a graph, if the degree of each vertex is ‘k’, then the graph is called a ‘k-regular graph’.

Example

In the following graphs, all the vertices have the same degree. So these graphs are called regular graphs.



In both the graphs, all the vertices have degree 2. They are called 2-Regular Graphs.

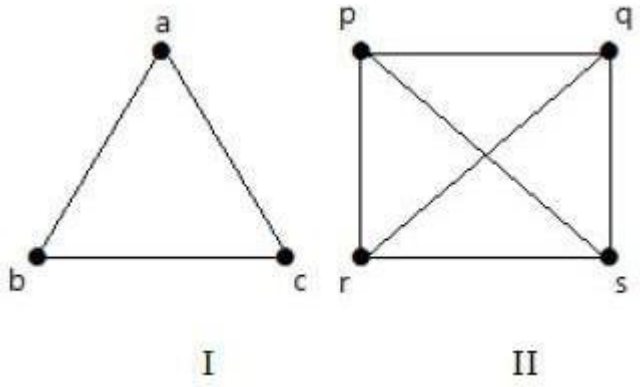
Complete Graph

A simple graph with 'n' mutual vertices is called a complete graph and it is **denoted by 'K_n'**. In the graph, **a vertex should have edges with all other vertices,** then it called a complete graph.

In other words, if a vertex is connected to all other vertices in a graph, then it is called a complete graph.

Example

In the following graphs, each vertex in the graph is connected with all the remaining vertices in the graph except by itself.



In graph I,

	a	b	c
a	Not Connected	Connected	Connected
b	Connected	Not Connected	Connected
c	Connected	Connected	Not Connected

In graph II,

	p	q	r	s
p	Not Connected	Connected	Connected	Connected
q	Connected	Not Connected	Connected	Connected

r	Connected	Connected	Not Connected	Connected
s	Connected	Connected	Connected	Not Connected

Cycle Graph

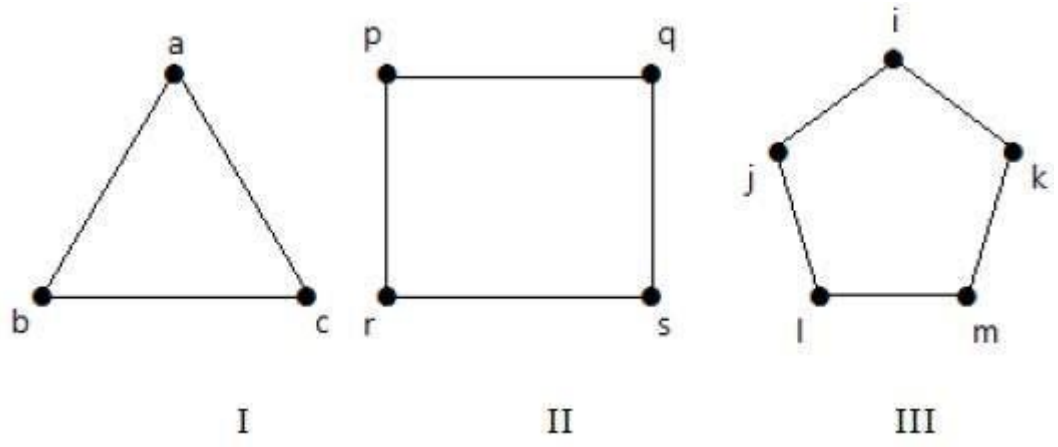
A simple graph with 'n' vertices $n >= 3$ and 'n' edges is called a cycle graph if all its edges form a cycle of length 'n'.

If the **degree of each vertex in the graph is two,** then it is called a Cycle Graph.

Example

Take a look at the following graphs −

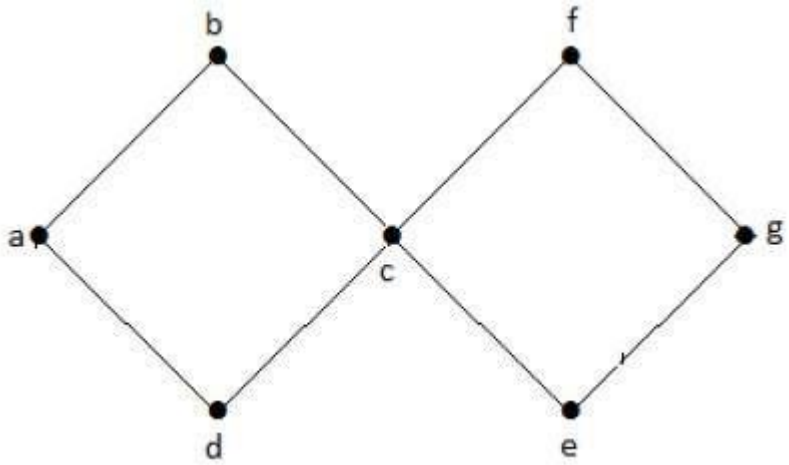
- Graph I has 3 vertices with 3 edges which is forming a cycle 'ab-bc-ca'.
- Graph II has 4 vertices with 4 edges which is forming a cycle 'pq-qs-sr-rp'.
- Graph III has 5 vertices with 5 edges which is forming a cycle 'ik-km-ml-lj-ji'.



Hence all the given graphs are cycle graphs.

Cyclic Graph

A graph **with at least one** cycle is called a cyclic graph.



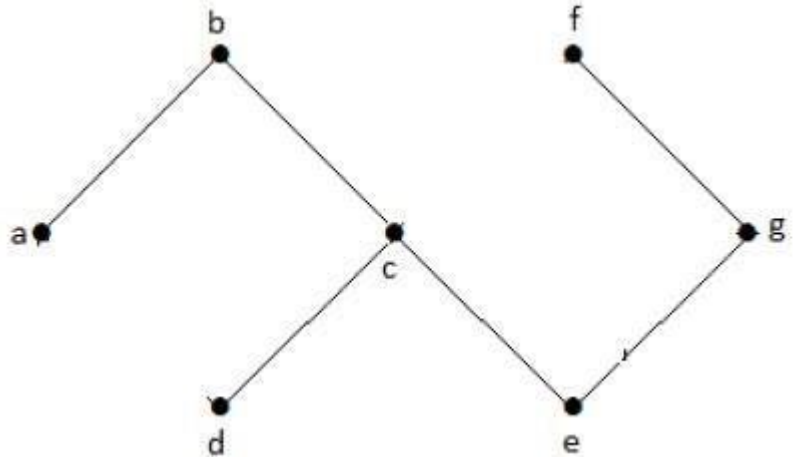
Example

In the above example graph, we have two cycles a-b-c-d-a and c-f-g-e-c. Hence it is called a cyclic graph.

Acyclic Graph

A graph **with no cycles** is called an acyclic graph.

In the above example graph, we do not have any cycles. Hence it is a non-cyclic graph.



Tree

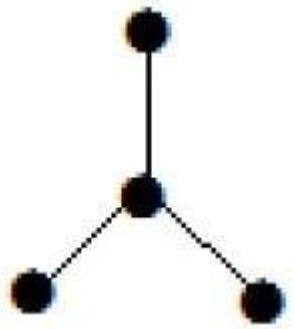
A ***connected acyclic graph*** is called a tree. In other words, a connected graph with no cycles is called a tree.

The edges of a tree are known as ***branches***. Elements of trees are called their ***nodes***. The nodes without child nodes are called ***leaf nodes***.

A tree with 'n' vertices has 'n−1' edges. If it has one more edge extra than 'n−1', then the extra edge should obviously has to pair up with two vertices which leads to form a cycle. Then, it becomes a cyclic graph which is a violation for the tree graph.

Example 1

The graph shown here is a tree because it has no cycles and it is connected. It has four vertices and three edges, i.e., for 'n' vertices 'n−1' edges as mentioned in the definition.



Note − Every tree has at least two vertices of degree one.

Example 2

In the above example, the vertices 'a' and 'd' has degree one. And the other two vertices 'b' and 'c' has degree two. This is possible because for not forming a cycle, there should be at least two single edges anywhere in the graph. It is nothing but two edges with a degree of one.

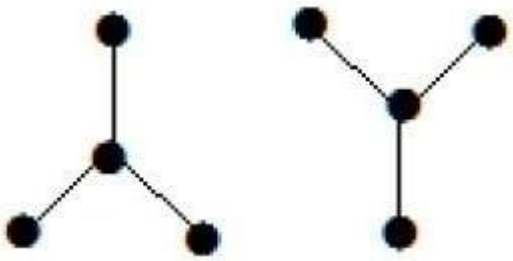
Forest

A ***disconnected acyclic graph*** is called a forest. In other words, a disjoint collection of trees is

called a forest.

Example

The following graph looks like two sub-graphs; but it is a single disconnected graph. There are no cycles in this graph. Hence, clearly it is a forest.



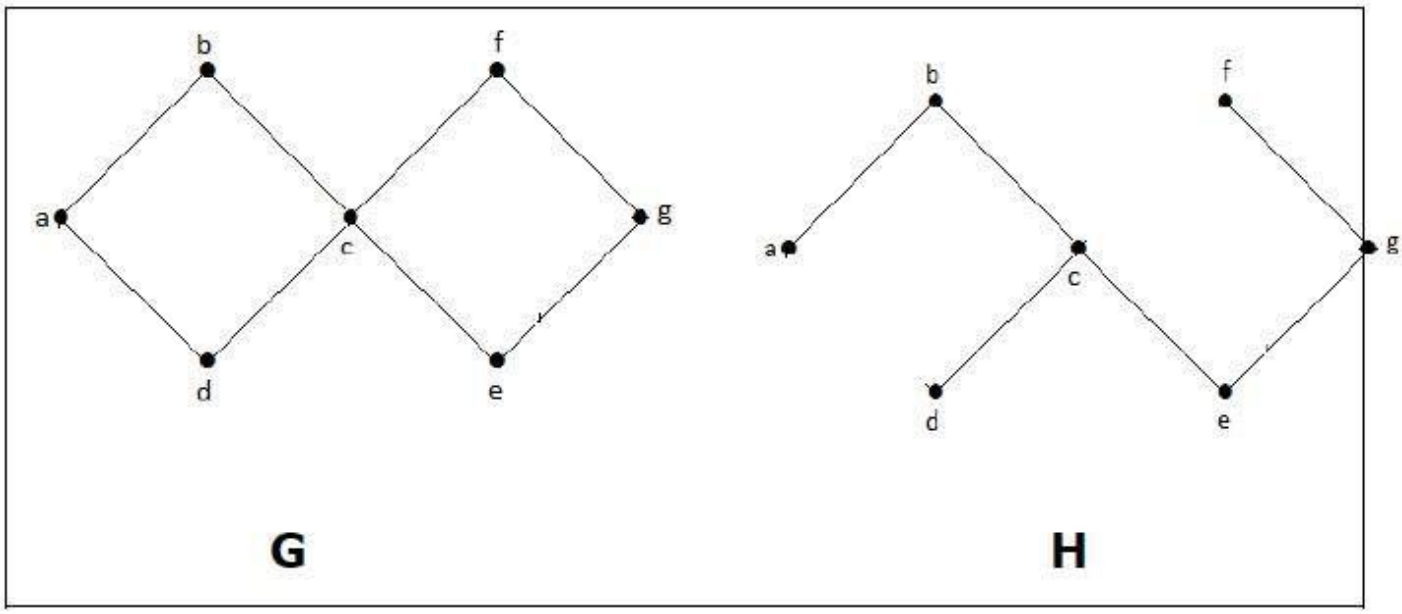
Spanning Trees

Let G be a connected graph, then the sub-graph H of G is called a spanning tree of G if −

- H is a tree
- H contains all vertices of G.

A spanning tree T of an undirected graph G is a subgraph that includes all of the vertices of G.

Example

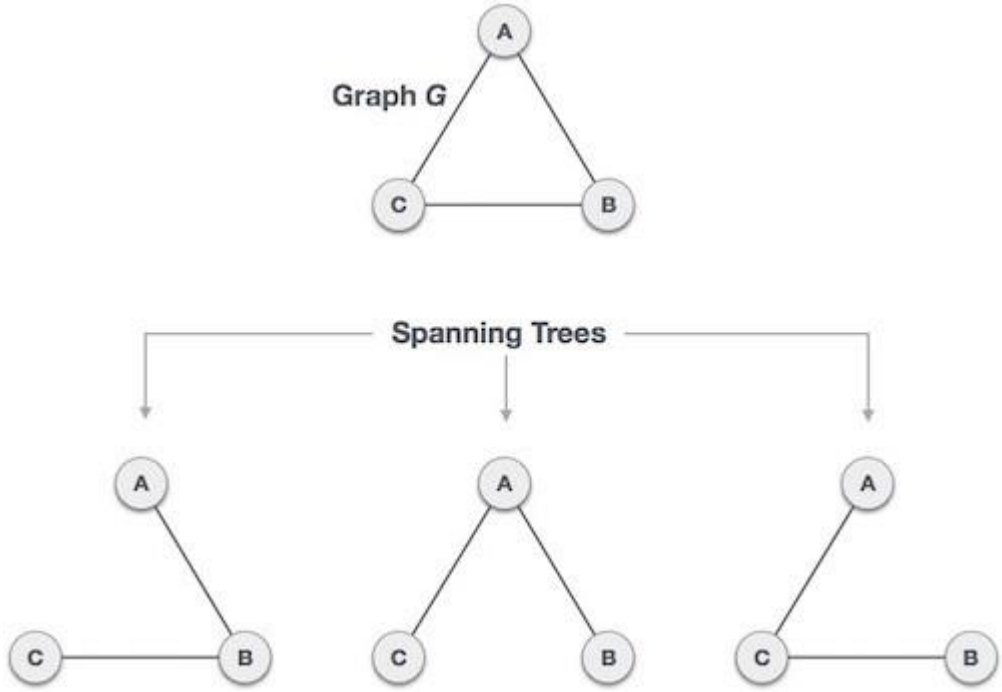


In the above example, G is a connected graph and H is a sub-graph of G.

Clearly, the graph H has no cycles, it is a tree with six edges which is one less than the total number of vertices. Hence H is the Spanning tree of G.

A spanning tree is a subset of Graph G, which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..

By this definition, we can draw a conclusion that every connected and undirected Graph G has at least one spanning tree. A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.



We found three spanning trees off one complete graph. A complete undirected graph can have maximum $\mathbf{n^{n-2}}$ number of spanning trees, where **n** is the number of nodes. In the above addressed example, **n is 3,** hence $\mathbf{3^{3-2} = 3}$ spanning trees are possible.

General Properties of Spanning Tree

We now understand that one graph can have more than one spanning tree. Following are a few properties of the spanning tree connected to graph G −

- A connected graph G can have more than one spanning tree.
- All possible spanning trees of graph G, have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops).
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.

Application of Spanning Tree

Spanning tree is basically used to find a minimum path to connect all nodes in a graph. Common application of spanning trees are −

- **Civil Network Planning**
- **Computer Network Routing Protocol**
- **Cluster Analysis**

Let us understand this through a small example. Consider, city network as a huge graph and now plans to deploy telephone lines in such a way that in minimum lines we can connect to all city

nodes. This is where the spanning tree comes into picture.

Minimum Spanning Tree (MST)

In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph. In real-world situations, this weight can be measured as distance, congestion, traffic load or any arbitrary value denoted to the edges.

Minimum Spanning-Tree Algorithm

We shall learn about two most important spanning tree algorithms here −

- Kruskal's Algorithm
- Prim's Algorithm

Both are greedy algorithms.

Graph Representations

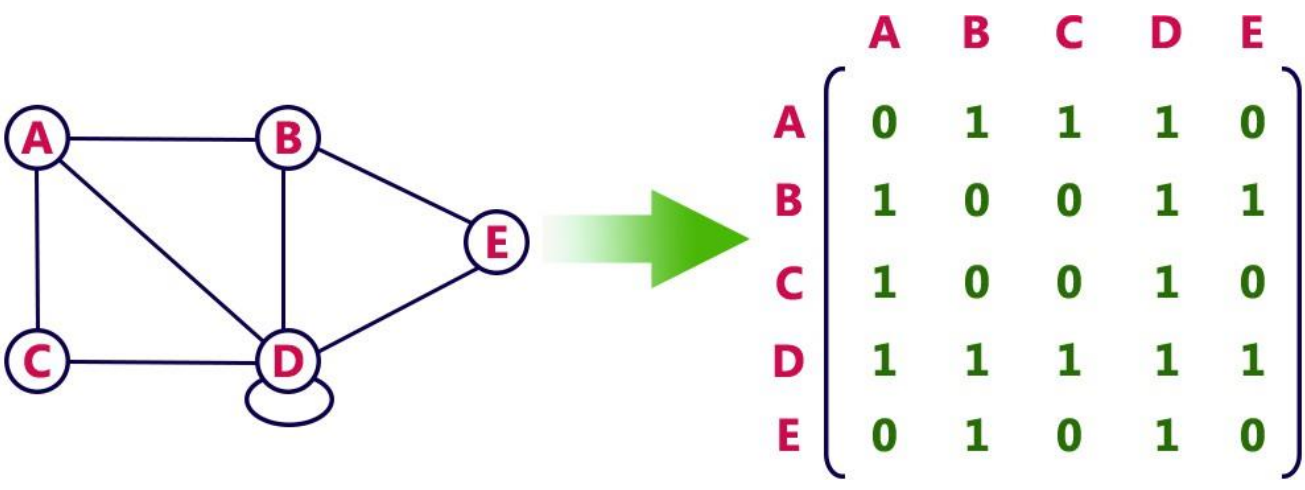
Graph data structure is represented using following representations...

1. Adjacency Matrix
2. Incidence Matrix
3. Adjacency List

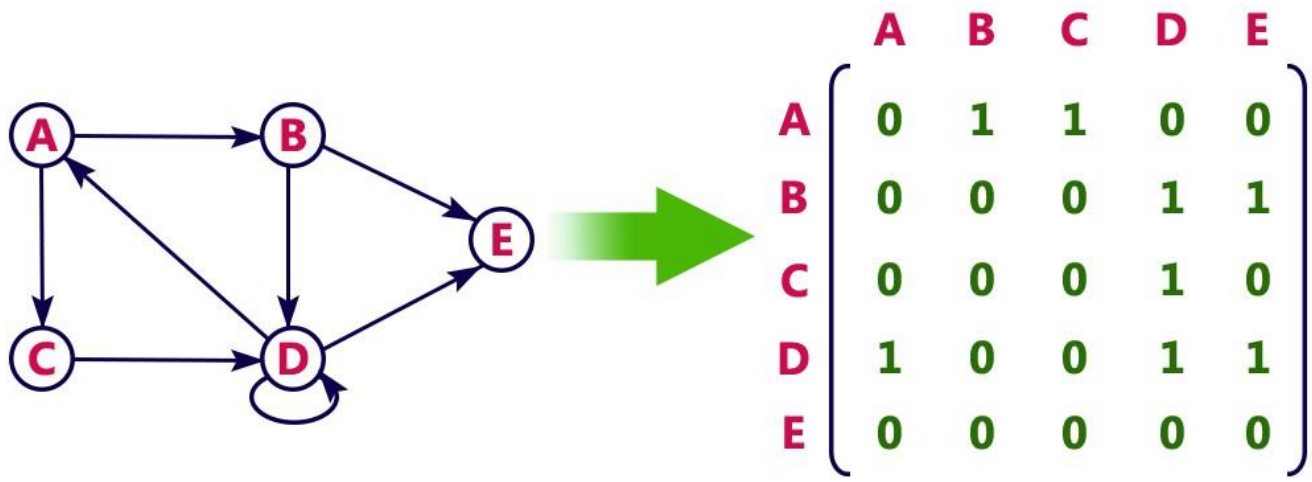
Adjacency Matrix

In this representation, the graph is represented using a matrix of size total number of vertices by a total number of vertices. That means a graph with 4 vertices is represented using a matrix of size 4X4. In this matrix, both rows and columns represent vertices. This matrix is filled with either 1 or 0. Here, 1 represents that there is an edge from row vertex to column vertex and 0 represents that there is no edge from row vertex to column vertex.

For example, consider the following undirected graph representation...



Directed graph representation...



Incidence Matrix

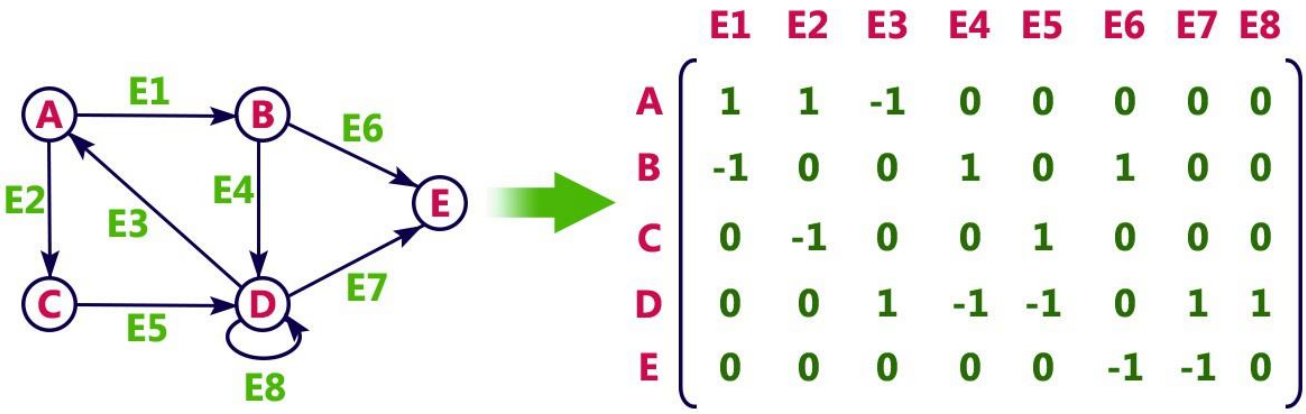
In this representation, the graph is represented using a matrix of size total number of vertices by a total number of edges. That means graph with 4 vertices and 6 edges is represented using a matrix of size 4X6. In this matrix, rows represent vertices and columns represents edges. This matrix is filled with 0 or 1 or -1.

Here, 0 represents that the row edge is not connected to column vertex,

1 represents that the row edge is connected as the outgoing edge to column vertex and

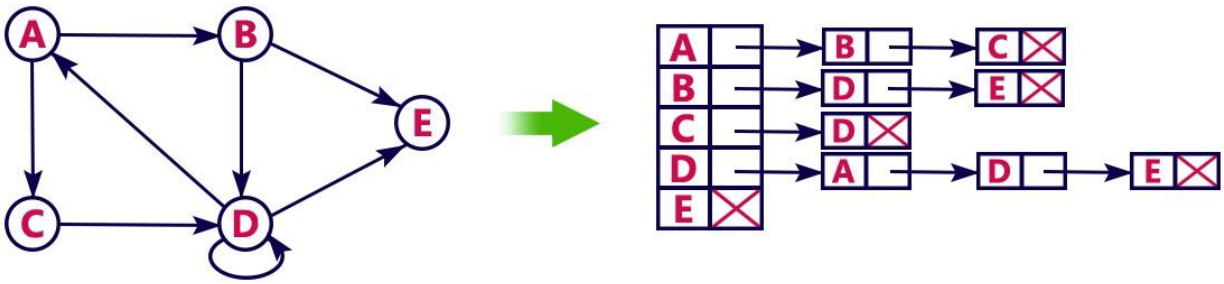
-1 represents that the row edge is connected as the incoming edge to column vertex.

For example, consider the following directed graph representation...

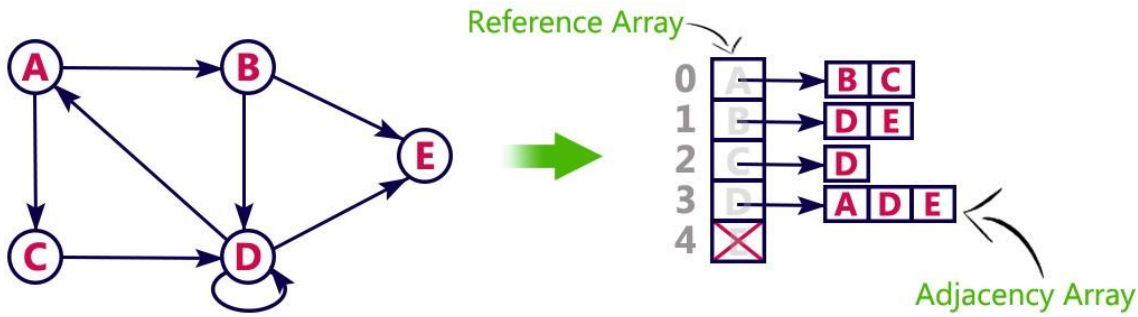


Adjacency List

In this representation, every vertex of a graph contains list of its adjacent vertices.
For example, consider the following directed graph representation implemented using linked list...



This representation can also be implemented using an array as follows..



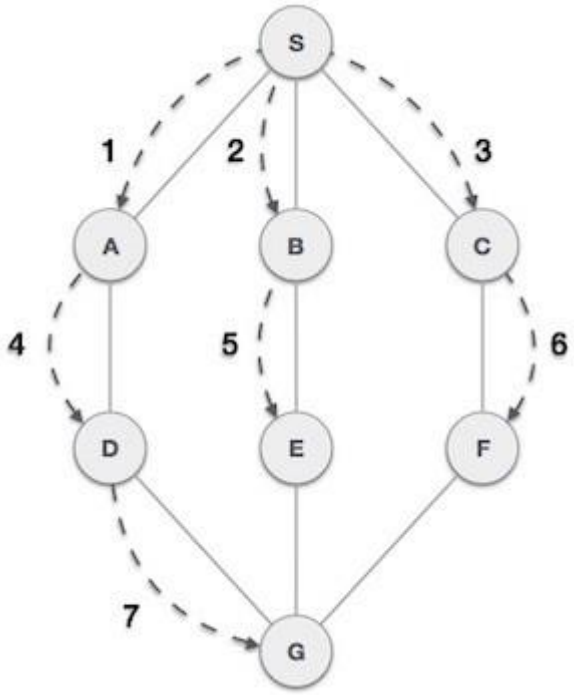
Graph Traversal

Graph traversal is a technique used for searching a vertex in a graph. The graph traversal is also used to decide the order of vertices is visited in the search process. A graph traversal finds the edges to be used in the search process without creating loops. That means using graph traversal we visit all the vertices of the graph without getting into looping path.

There are two graph traversal techniques and they are as follows...

1. **BFS (Breadth First Search)**
2. **DFS (Depth First Search)**

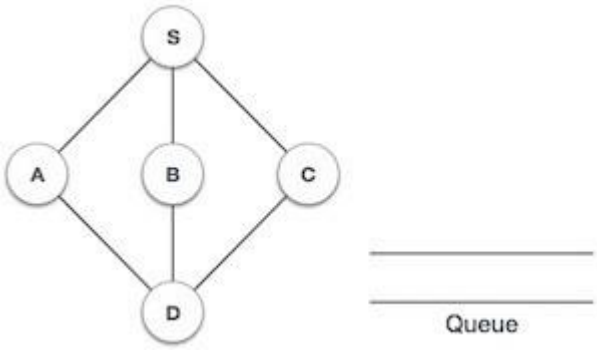
Breadth First Search (BFS)

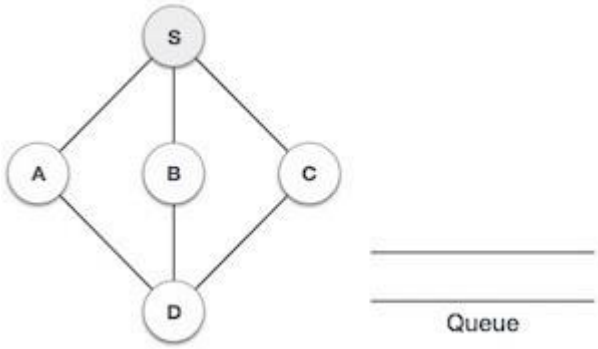
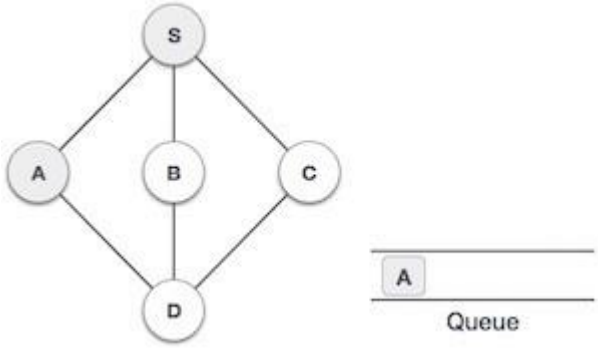
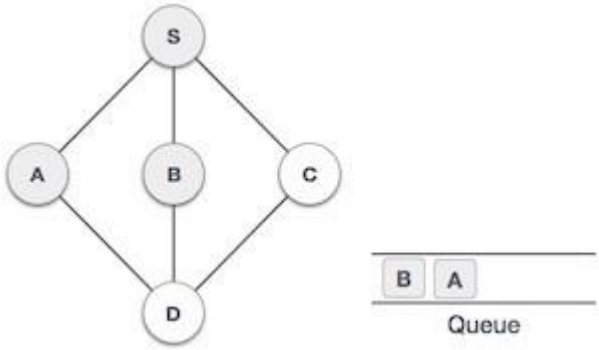
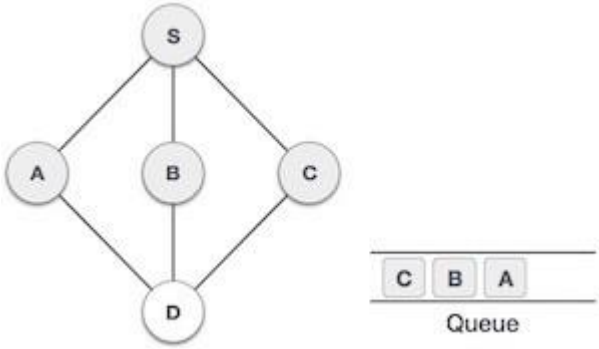
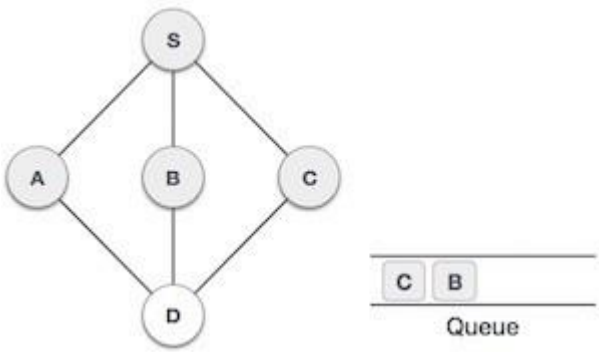


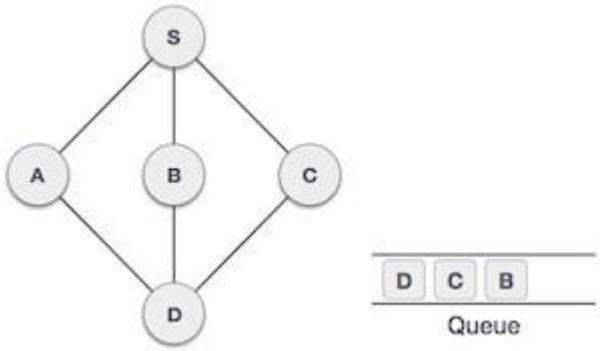
Breadth First Search (BFS) algorithm traverses a graph in a breadthward motion and uses a queue to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

As in the example given above, BFS algorithm traverses from A to B to E to F first then to C and G lastly to D. It employs the following rules.

- Rule 1 − Visit the adjacent unvisited vertex. Mark it as visited. Display it. Insert it in a queue.
- Rule 2 − If no adjacent vertex is found, remove the first vertex from the queue.
- Rule 3 − Repeat Rule 1 and Rule 2 until the queue is empty.

Step	Traversal	Description
1		Initialize the queue.

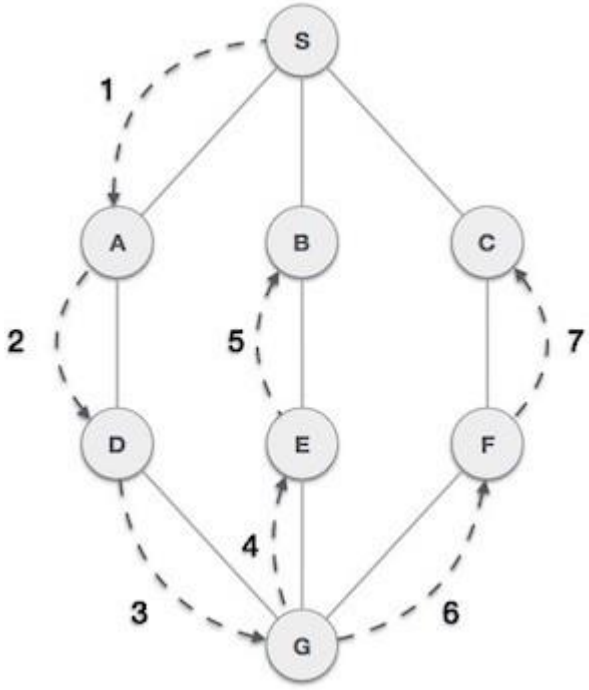
2		We start from visiting S (starting node), and mark it as visited.
3		We then see an unvisited adjacent node from S. In this example, we have three nodes but alphabetically we choose A, mark it as visited and enqueue it.
4		Next, the unvisited adjacent node from S is B. We mark it as visited and enqueue it.
5		Next, the unvisited adjacent node from S is C. We mark it as visited and enqueue it.
6		Now, S is left with no unvisited adjacent nodes. So, we dequeue A.

7		From A we have D as unvisited adjacent node. We mark it as visited and enqueue it.

At this stage, we are left with no unmarked (unvisited) nodes. But as per the algorithm we keep on dequeuing in order to get all unvisited nodes. When the queue gets emptied, the program is over.

Depth First Traversal

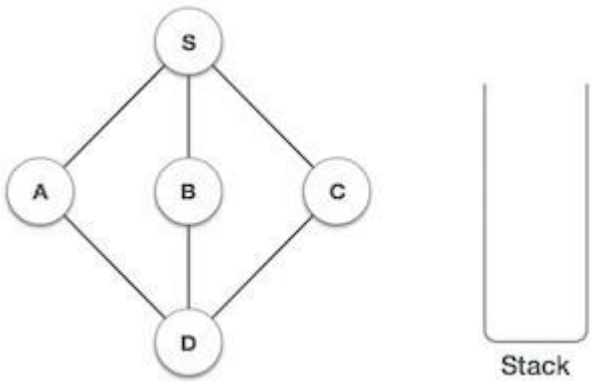
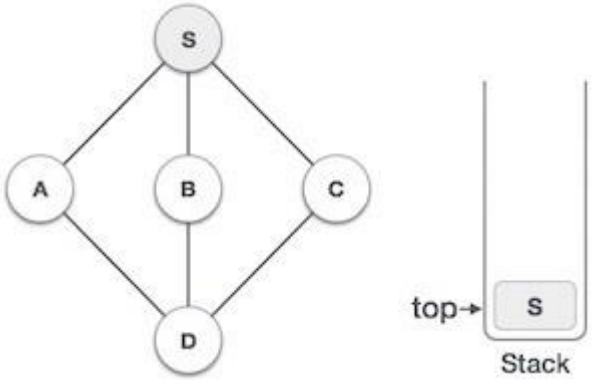
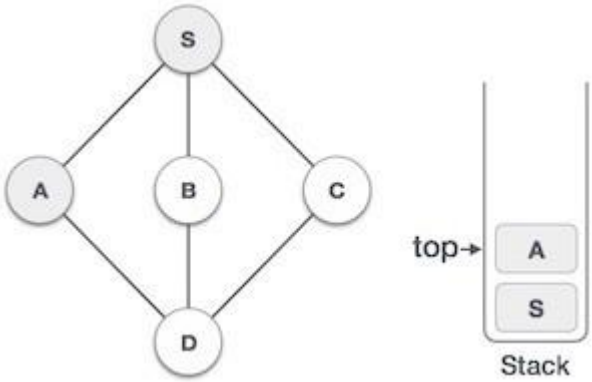
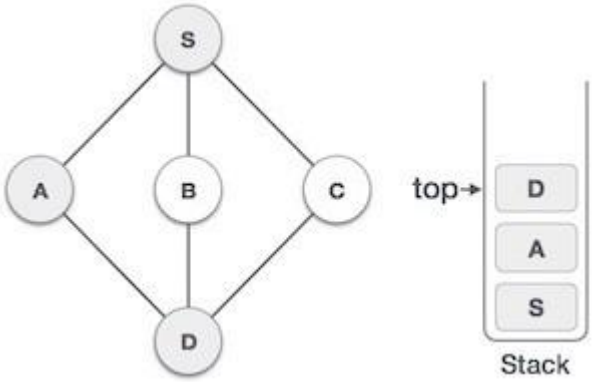
Depth First Search (DFS) algorithm traverses a graph in a depthward motion and uses a stack to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

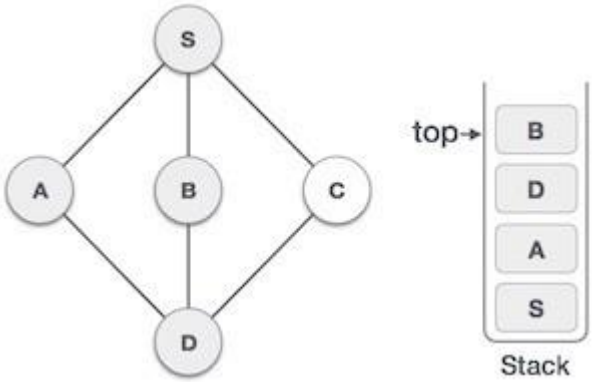
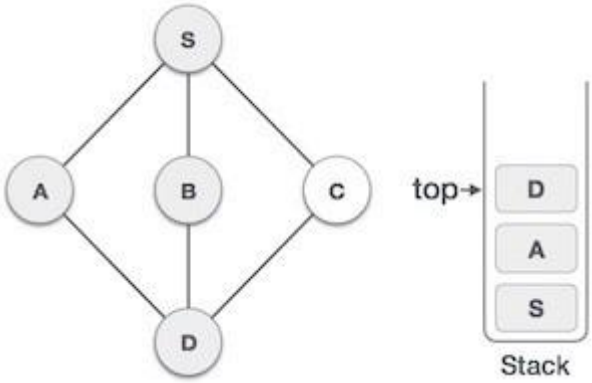
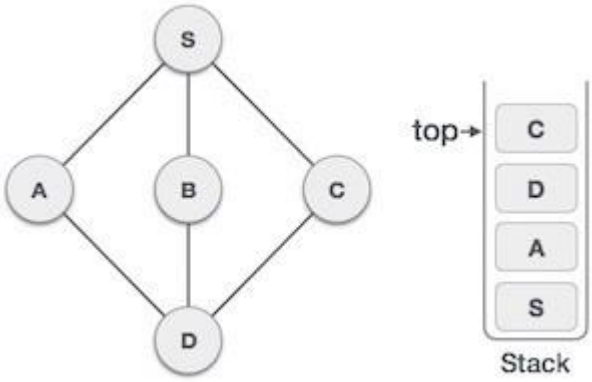


As in the example given above, DFS algorithm traverses from S to A to D to G to E to B first, then to F and lastly to C. It employs the following rules.

- **Rule 1** − Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.
- **Rule 2** − If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)
- **Rule 3** − Repeat Rule 1 and Rule 2 until the stack is empty.

Step	Traversal	Description

1		Initialize the stack.
2		Mark **S** as visited and put it onto the stack. Explore any unvisited adjacent node from **S**. We have three nodes and we can pick any of them. For this example, we shall take the node in an alphabetical order.
3		Mark **A** as visited and put it onto the stack. Explore any unvisited adjacent node from A. Both **S** and **D** are adjacent to **A** but we are concerned for unvisited nodes only.
4		Visit **D** and mark it as visited and put onto the stack. Here, we have **B** and **C** nodes, which are adjacent to **D** and both are unvisited. However, we shall again choose in an alphabetical order.
5		We choose **B**, mark it as visited and put onto the stack. Here **B** does not have any unvisited adjacent node. So, we pop **B** from the stack.

		
6		We check the stack top for return to the previous node and check if it has any unvisited nodes. Here, we find **D** to be on the top of the stack.
7		Only unvisited adjacent node is from **D** is **C** now. So we visit **C**, mark it as visited and put it onto the stack.

BFS (Breadth First Search)

BFS traversal of a graph produces a **spanning tree** as final result. **Spanning Tree** is a graph without loops. We use **Queue data structure** with maximum size of total number of vertices in the graph to implement BFS traversal.

We use the following steps to implement BFS traversal...

Step 1 - Define a Queue of size total number of vertices in the graph.

Step 2 - Select any vertex as **starting point** for traversal. Visit that vertex and insert it into the Queue.

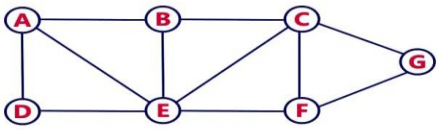
Step 3 - Visit all the non-visited **adjacent** vertices of the vertex which is at front of the Queue and insert them into the Queue.

Step 4 - When there is no new vertex to be visited from the vertex which is at front of the Queue then delete that vertex.

Step 5 - Repeat steps 3 and 4 until queue becomes empty.

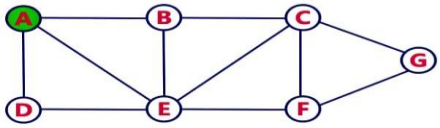
Step 6 - When queue becomes empty, then produce final spanning tree by removing unused edges from the graph

Consider the following example graph to perform BFS traversal



Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.

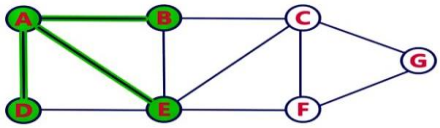


Queue

A						

Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D**, **E**, **B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..

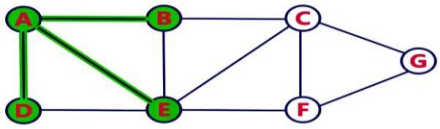


Queue

	D	E	B			

Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.

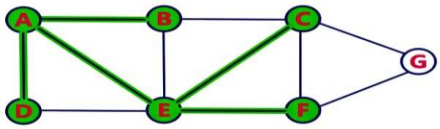


Queue

		E	B			

Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C**, **F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.

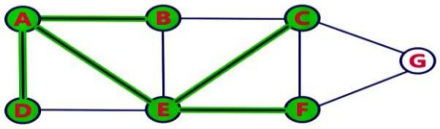


Queue

			B	C	F	

Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.

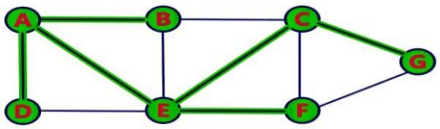


Queue

				C	F	

Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.

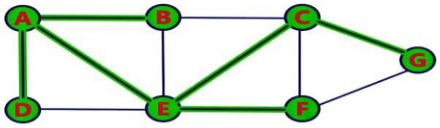


Queue

					F	G

Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.

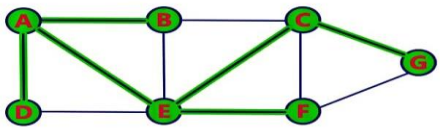


Queue

						G

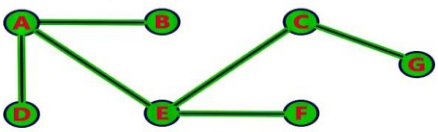
Step 8:

- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.



Queue

- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



DFS (Depth First Search)

DFS traversal of a graph produces a **spanning tree** as final result. **Spanning Tree** is a graph without loops. We use **Stack data structure** with maximum size of total number of vertices in the graph to implement DFS traversal.

We use the following steps to implement DFS traversal...

Step 1 - Define a Stack of size total number of vertices in the graph.

Step 2 - Select any vertex as **starting point** for traversal. Visit that vertex and push it on to the Stack.

Step 3 - Visit any one of the non-visited **adjacent** vertices of a vertex which is at the top of stack and push it on to the stack.

Step 4 - Repeat step 3 until there is no new vertex to be visited from the vertex which is at the top of the stack.

Step 5 - When there is no new vertex to visit then use **back tracking** and pop one vertex from the stack.

Step 6 - Repeat steps 3, 4 and 5 until stack becomes Empty.

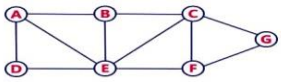
Step 7 - When stack becomes Empty, then produce final spanning tree by removing unused edges from the graph

---

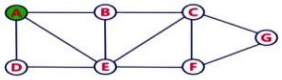
Back tracking is coming back to the vertex from which we reached the current vertex.

Example

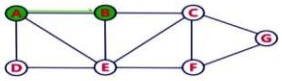
Consider the following example graph to perform DFS traversal



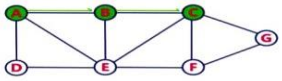
Step 1:
- Select the vertex **A** as starting point (visit **A**).
- Push **A** on to the Stack.

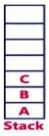


Step 2:
- Visit any adjacent vertex of **A** which is not visited (**B**).
- Push newly visited vertex B on to the Stack.

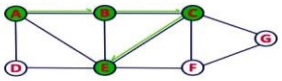


Step 3:
- Visit any adjacent vertext of **B** which is not visited (**C**).
- Push C on to the Stack.

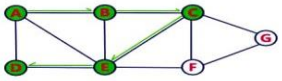




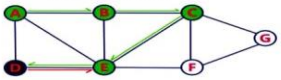
Step 4:
- Visit any adjacent vertext of **C** which is not visited (**E**).
- Push E on to the Stack

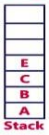


Step 5:
- Visit any adjacent vertext of **E** which is not visited (**D**).
- Push D on to the Stack

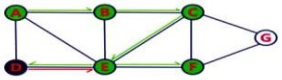


Step 6:
- There is no new vertiex to be visited from D. So use back track.
- Pop D from the Stack.

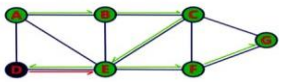




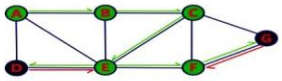
Step 7:
- Visit any adjacent vertex of **E** which is not visited (**F**).
- Push F on to the Stack.



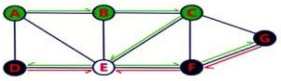
Step 8:
- Visit any adjacent vertex of **F** which is not visited (**G**).
- Push G on to the Stack.

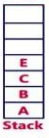


Step 9:
- There is no new vertiex to be visited from G. So use back track.
- Pop G from the Stack.

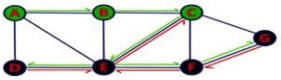


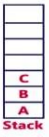
Step 10:
- There is no new vertiex to be visited from F. So use back track.
- Pop F from the Stack.



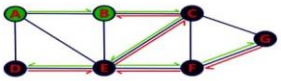


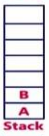
Step 11:
- There is no new vertiex to be visited from E. So use back track.
- Pop E from the Stack.





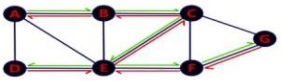
Step 12:
- There is no new vertiex to be visited from C. So use back track.
- Pop C from the Stack.



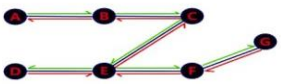


Step 13:
- There is no new vertiex to be visited from B. So use back track.
- Pop B from the Stack.

Step 14:
- There is no new vertiex to be visited from A. So use back track.
- Pop A from the Stack.



- Stack became Empty. So stop DFS Treversal.
- Final result of DFS traversal is following spanning tree.

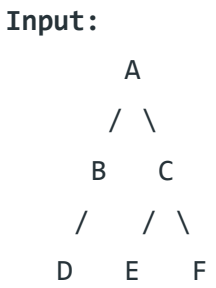


Difference between BFS and DFS

Breadth-First Search:

BFS stands for **Breadth-First Search** is a vertex-based technique for finding the shortest path in the graph. It uses a Queue data structure that follows first in first out. In BFS, one vertex is selected at a time when it is visited and marked then its adjacent are visited and stored in the queue. It is slower than DFS.

Example:



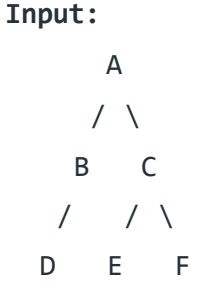
Output:

A, B, C, D, E, F

Depth First Search:

DFS stands for **Depth First Search** is an edge-based technique. It uses the Stack data structure and performs two stages, first visited vertices are pushed into the stack, and second if there are no vertices then visited vertices are popped.

Example:



Output:

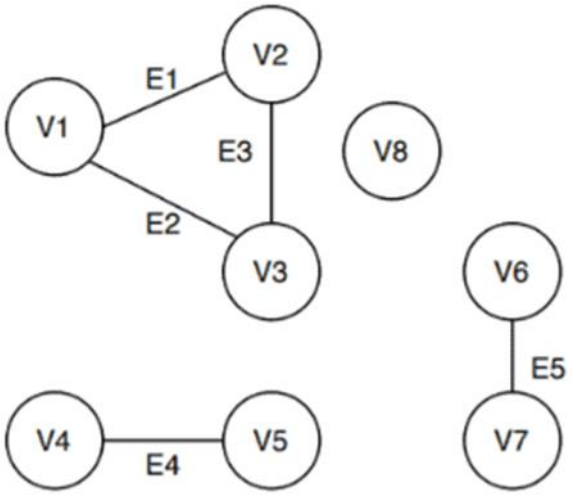
A, B, D, C, E, F

BFS vs DFS

S. No.	Parameters	BFS	DFS
1.	Stands for	BFS stands for Breadth First Search.	DFS stands for Depth First Search.
2.	Data	BFS(Breadth First Search) uses Queue data	DFS(Depth First Search) uses Stack data structure.

	Structure	structure for finding the shortest path.	
3.	Definition	BFS is a traversal approach in which we first walk through all nodes on the same level before moving on to the next level.	DFS is also a traversal approach in which the traverse begins at the root node and proceeds through the nodes as far as possible until we reach the node with no unvisited nearby nodes.
4.	Technique	BFS can be used to find a single source shortest path in an unweighted graph because, in BFS, we reach a vertex with a minimum number of edges from a source vertex.	In DFS, we might traverse through more edges to reach a destination vertex from a source.
5.	Conceptual Difference	BFS builds the tree level by level.	DFS builds the tree sub-tree by sub-tree.
6.	Approach used	It works on the concept of FIFO (First In First Out).	It works on the concept of LIFO (Last In First Out).
7.	Suitable for	BFS is more suitable for searching vertices which are closer to the given source.	DFS is more suitable when there are solutions away from source.
8.	Suitable for Decision Tress	BFS considers all neighbors first and therefore not suitable for decision making trees used in games or puzzles.	DFS is more suitable for game or puzzle problems. We make a decision, then explore all paths through this decision. And if this decision leads to win situation, we stop.
9.	Time Complexity	The Time complexity of BFS is O(V + E) when Adjacency List is used and O(V^2) when Adjacency Matrix is used, where V stands for vertices and E stands for edges.	The Time complexity of DFS is also O(V + E) when Adjacency List is used and O(V^2) when Adjacency Matrix is used, where V stands for vertices and E stands for edges.
10.	Visiting of Siblings/ Children	Here, siblings are visited before the children.	Here, children are visited before the siblings.
11.	Removal of Traversed Nodes	Nodes that are traversed several times are deleted from the queue.	The visited nodes are added to the stack and then removed when there are no more nodes to visit.
12.	Backtracking	In BFS there is no concept of backtracking.	DFS algorithm is a recursive algorithm that uses the idea of backtracking
13.	Applications	BFS is used in various application such as bipartite graph, and shortest path etc.	DFS is used in various application such as acyclic graph and topological order etc.
14.	Memory	BFS requires more memory.	DFS requires less memory.
15.	Optimality	BFS is optimal for finding the shortest path.	DFS is not optimal for finding the shortest path.
16.	Space complexity	In BFS, the space complexity is more critical as compared to time complexity.	DFS has lesser space complexity, because at a time it needs to store only single path from the root to leaf node.
17.	Speed	BFS is slow as compared to DFS.	DFS is fast as compared to BFS.
18.	When to use?	When the target is close to the source, BFS performs better.	When the target is far from the source, DFS is preferable.

Connected components of a graph



Connected components of a graph are subsets of vertices such that each subset forms a subgraph where there is a path between any pair of vertices, and which is not connected to any other vertex in the larger graph.
To find the connected components of an undirected graph, we can use Depth-First Search (DFS) or Breadth-First Search (BFS).

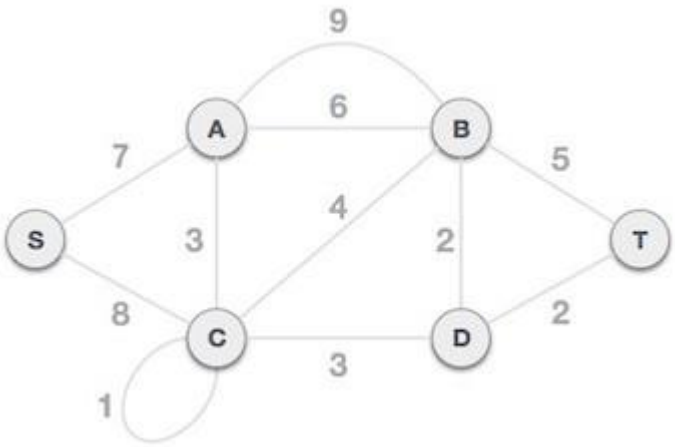
Below is the algorithm for finding the connected components using DFS.
Algorithm for Finding Connected Components Using DFS

1. Initialization:
    - Create a list visited to keep track of visited vertices.
    - Create a list components to store all the connected components.
    - Initialize all vertices in visited as False.
2. Traversal:
    - For each vertex v in the graph:
        - If v is not visited, perform a DFS starting from v to find all vertices in the same connected component.
        - Mark all these vertices as visited.
        - Add the discovered connected component to the components list.
3. DFS Function:
    - The DFS function recursively explores all vertices connected to a given vertex.

Kruskal's Minimum Spanning Tree(MST) Algorithm

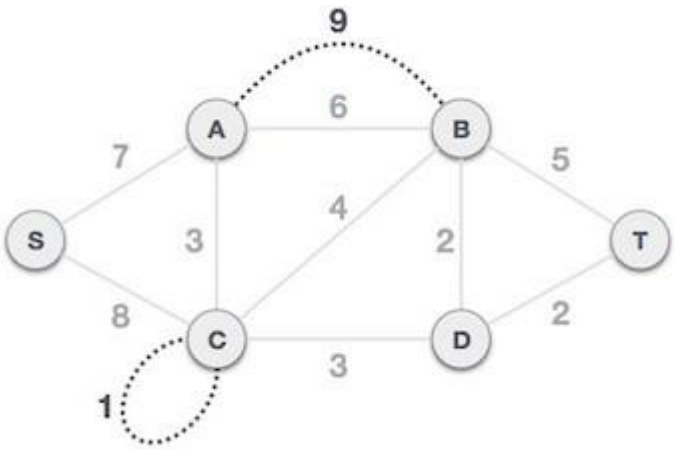
Kruskal's algorithm to find the minimum cost spanning tree uses the greedy approach. This algorithm treats the graph as a forest and every node it has as an individual tree. A tree connects to another only and only if, it has the least cost among all available options and does not violate MST properties.

To understand Kruskal's algorithm let us consider the following example −

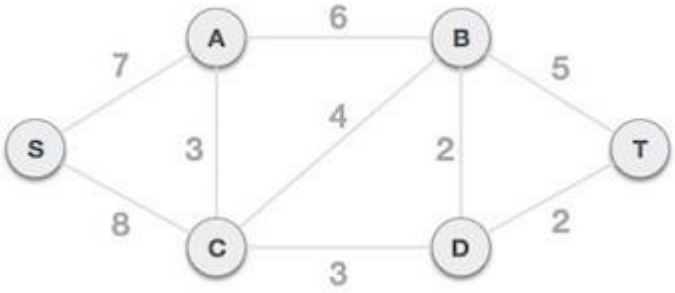


Step 1 - Remove all loops and Parallel Edges

Remove all loops and parallel edges from the given graph.



In case of parallel edges, keep the one which has the least cost associated and remove all others.



Step 2 - Arrange all edges in their increasing order of weight

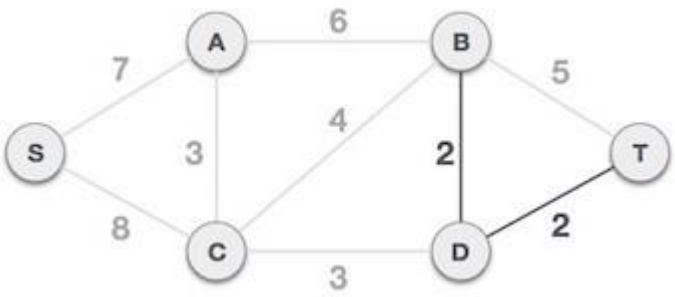
The next step is to create a set of edges and weight, and arrange them in an ascending order of weightage (cost).

B, D	D, T	A, C	C, D	C, B	B, T	A, B	S, A	S, C
2	2	3	3	4	5	6	7	8

Step 3 - Add the edge which has the least weightage

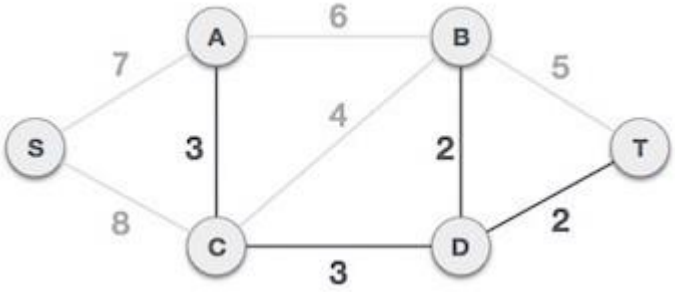
Now we start adding edges to the graph beginning from the one which has the least weight. Throughout, we shall keep checking that the spanning properties remain intact. In case, by adding

one edge, the spanning tree property does not hold then we shall consider not to include the edge in the graph.

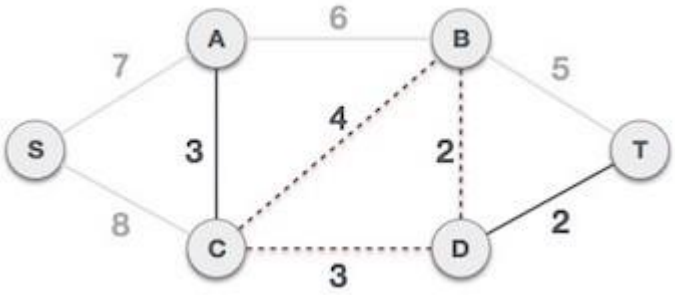


The least cost is 2 and edges involved are B,D and D,T. We add them. Adding them does not violate spanning tree properties, so we continue to our next edge selection.

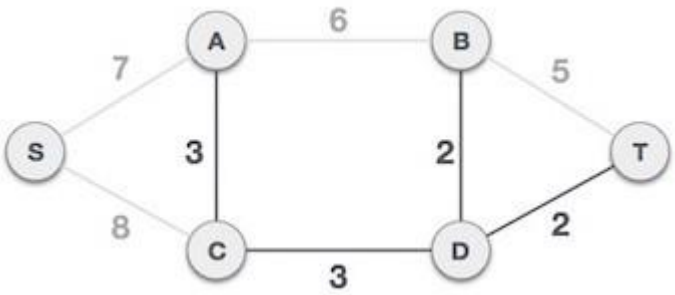
Next cost is 3, and associated edges are A,C and C,D. We add them again −



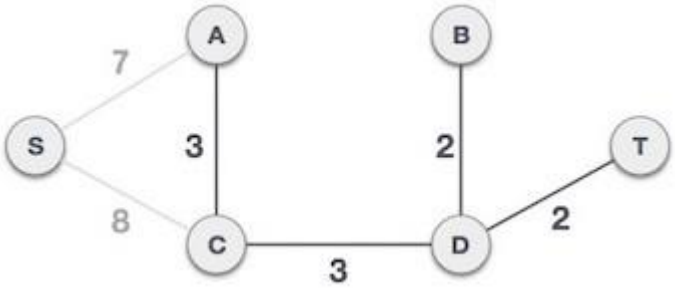
Next cost in the table is 4, and we observe that adding it will create a circuit in the graph. −



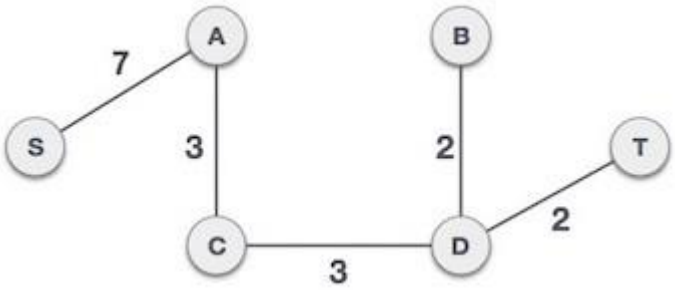
We ignore it. In the process we shall ignore/avoid all edges that create a circuit.



We observe that edges with cost 5 and 6 also create circuits. We ignore them and move on.



Now we are left with only one node to be added. Between the two least cost edges available 7 and 8, we shall add the edge with cost 7.



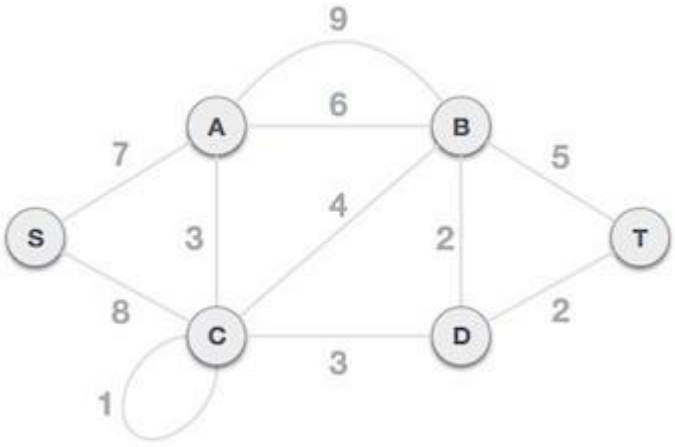
By adding edge S,A we have included all the nodes of the graph and we now have minimum cost spanning tree.

Prim's Minimum Spanning Tree (MST) Algorithm

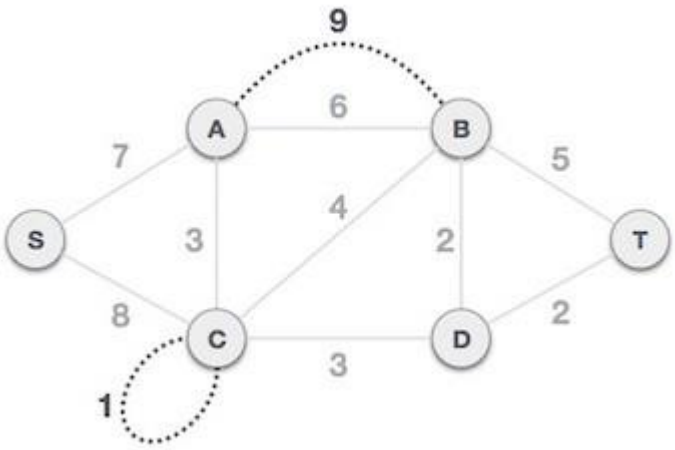
Prim's algorithm to find minimum cost spanning tree (as Kruskal's algorithm) uses the greedy approach. Prim's algorithm shares a similarity with the **shortest path first** algorithms.

Prim's algorithm, in contrast with Kruskal's algorithm, treats the nodes as a single tree and keeps on adding new nodes to the spanning tree from the given graph.

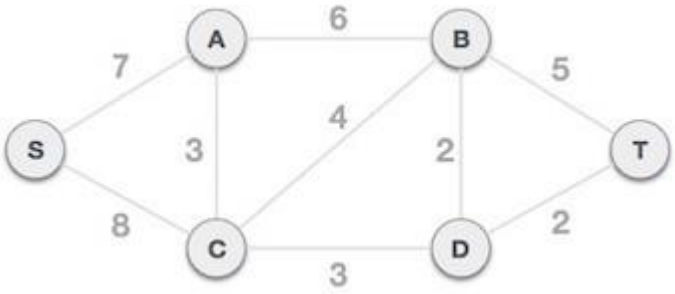
To contrast with Kruskal's algorithm and to understand Prim's algorithm better, we shall use the same example −



Step 1 - Remove all loops and parallel edges



Remove all loops and parallel edges from the given graph. In case of parallel edges, keep the one which has the least cost associated and remove all others.

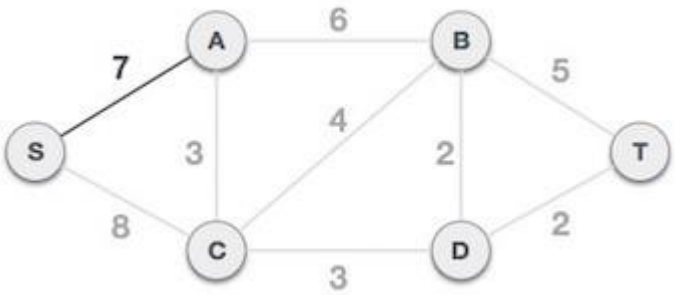


Step 2 - Choose any arbitrary node as root node

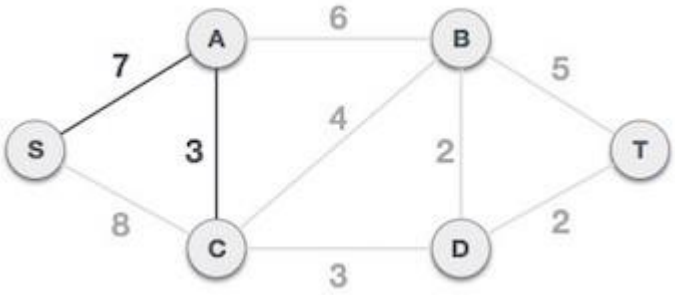
In this case, we choose **S** node as the root node of Prim's spanning tree. This node is arbitrarily chosen, so any node can be the root node. One may wonder why any video can be a root node. So the answer is, in the spanning tree all the nodes of a graph are included and because it is connected then there must be at least one edge, which will join it to the rest of the tree.

Step 3 - Check outgoing edges and select the one with less cost

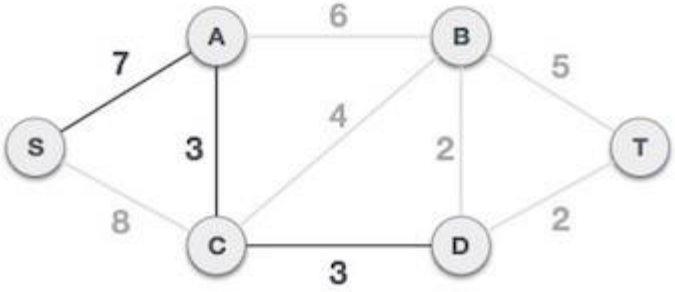
After choosing the root node **S**, we see that S,A and S,C are two edges with weight 7 and 8, respectively. We choose the edge S,A as it is lesser than the other.



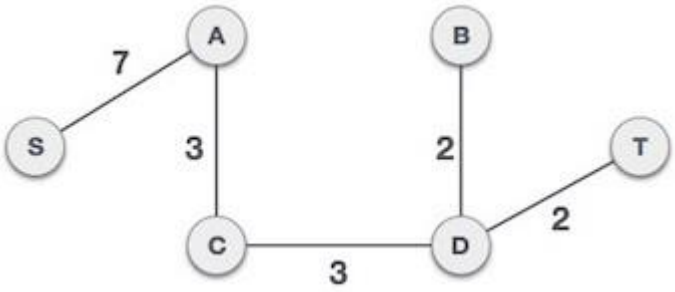
Now, the tree S-7-A is treated as one node and we check for all edges going out from it. We select the one which has the lowest cost and include it in the tree.



After this step, S-7-A-3-C tree is formed. Now we'll again treat it as a node and will check all the edges again. However, we will choose only the least cost edge. In this case, C-3-D is the new edge, which is less than other edges' cost 8, 6, 4, etc.



After adding node **D** to the spanning tree, we now have two edges going out of it having the same cost, i.e. D-2-T and D-2-B. Thus, we can add either one. But the next step will again yield edge 2 as the least cost. Hence, we are showing a spanning tree with both edges included.



We may find that the output spanning tree of the same graph using two different algorithms is same.

Difference between Prim's and Kruskal's algorithm for MST

Kruskal's algorithm for MST

Given a connected and undirected graph, a spanning tree of that graph is a subgraph that is a tree and connects all the vertices together. A single graph can have many different spanning trees. A minimum spanning tree (MST) or minimum weight spanning tree for a weighted, connected and undirected graph is a spanning tree with weight less than or equal to the weight of every other spanning tree. The weight of a spanning tree is the sum of weights given to each edge of the spanning tree.

Below are the steps for finding MST using Kruskal's algorithm

1. Sort all the edges in non-decreasing order of their weight.

2. Pick the smallest edge. Check if it forms a cycle with the spanning-tree formed so far. If the cycle is not formed, include this edge. Else, discard it.
3. Repeat step#2 until there are (V-1) edges in the spanning tree.

Prim's algorithm for MST

Like Kruskal's algorithm, Prim's algorithm is also a Greedy algorithm. It starts with an empty spanning tree. The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, the other set contains the vertices not yet included. At every step, it considers all the edges that connect the two sets and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST.

Below are the steps for finding MST using Prim's algorithm

1. Create a set mstSet that keeps track of vertices already included in MST.
2. Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE. Assign key value as 0 for the first vertex so that it is picked first.
3. While mstSet doesn't include all vertices
    - Pick a vertex u which is not there in mstSet and has minimum key value.
    - Include u to mstSet.
    - Update the key value of all adjacent vertices of u. To update the key values, iterate through all adjacent vertices. For every adjacent vertex v, if the weight of edge u-v is less than the previous key value of v, update the key value as the weight of u-v

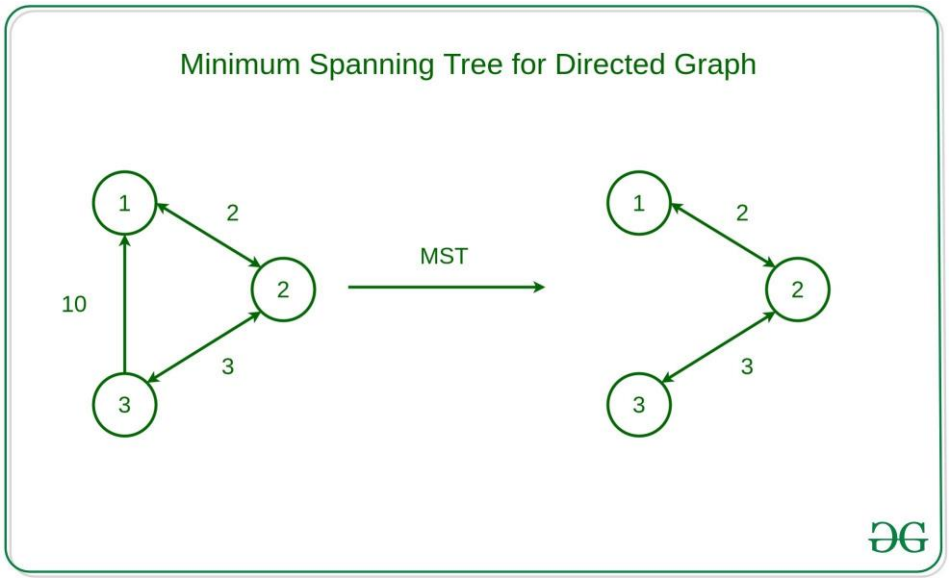
Both **Prim's and Kruskal's algorithm** finds the Minimum Spanning Tree and follow the Greedy approach of problem-solving, but there are few **major differences between them**.

Prim's Algorithm	Kruskal's Algorithm
It starts to build the Minimum Spanning Tree from any vertex in the graph.	It starts to build the Minimum Spanning Tree from the vertex carrying minimum weight in the graph.
It traverses one node more than one time to get the minimum distance.	It traverses one node only once.
Prim's algorithm has a time complexity of $O(V^2)$, V being the number of vertices and can be improved up to O(E log V) using Fibonacci heaps.	Kruskal's algorithm's time complexity is O(E log V), V being the number of vertices.
Prim's algorithm gives connected component as well as it works only on connected graph.	Kruskal's algorithm can generate forest(disconnected components) at any instant as well as it can work on disconnected components
Prim's algorithm runs faster in dense graphs.	Kruskal's algorithm runs faster in sparse graphs.
It generates the minimum spanning tree starting from the root vertex.	It generates the minimum spanning tree starting from the least weighted edge.
Applications of prim's algorithm are Travelling Salesman Problem, Network for roads and Rail tracks connecting all the cities etc.	Applications of Kruskal algorithm are LAN connection, TV Network etc.
Prim's algorithm prefer list data structures.	Kruskal's algorithm prefer heap data structures.

Why Prim's and Kruskal's MST algorithms fail for Directed Graphs?

Given a directed graph **D = < V, E >**, the task is to find the minimum spanning tree for the given directed graph

Example:

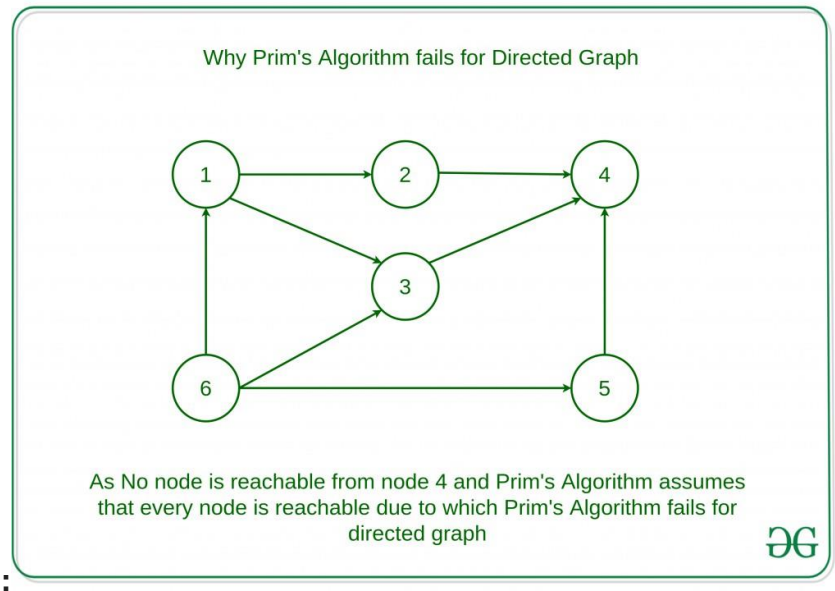


But the Prim's Minimum Spanning Tree and Kruskal's algorithm fails for directed graphs. Let us see why

Why Prim's Algorithm Fails for Directed Graph ?

Prim's algorithm assumes that all vertices are connected. But in a directed graph, every node is not reachable from every other node. So, Prim's algorithm fails due to this reason.

For Example:



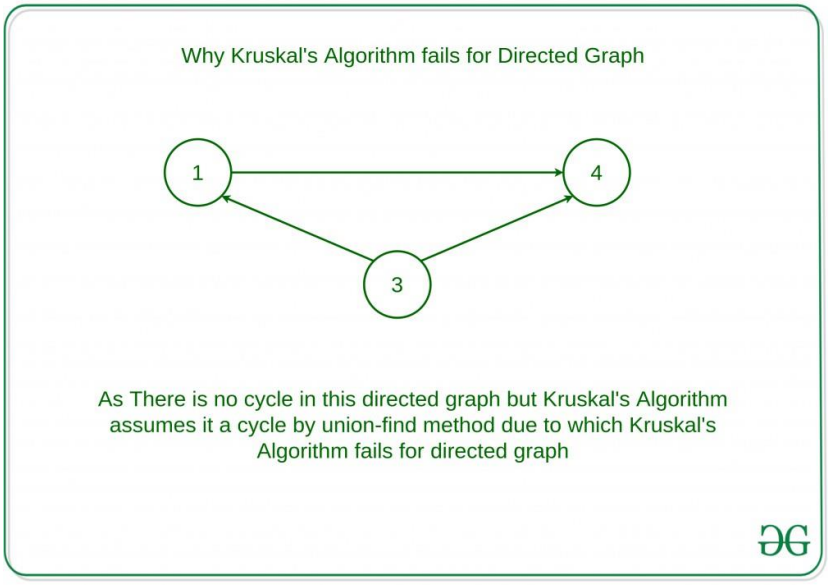
As it is visible in the graph, no node is reachable from node 4. Directed graphs fail the requirement that all vertices are connected.

Why Kruskal's Algorithm fails for directed graph?

In Kruskal's algorithm, In each step, it is checked that if the edges form a cycle with the spanning-tree formed so far. But Kruskal's algorithm fails to detect the cycles in a directed

graph as there are cases when there is no cycle between the vertices but Kruskal's Algorithm assumes it to cycle and don't take consider some edges due to which Kruskal's Algorithm fails for directed graph.

For example:



This graph will be reported to contain a cycle with the Union-Find method, but this graph has no cycle.

The equivalent of minimum spanning tree in directed graphs is, "Minimum Spanning Arborescence"(also known as optimum branching) can be solved by Edmonds' algorithm with a running time of O(EV). This algorithm is directed analog of the minimum spanning tree problem.